使 用 手 册

无人机高光谱张量农情分析系统

AXU5EV-E 板端使用手册 · 启动与烧录

项目	内容
文档编号	HSI-UM-002
版本 / 日期	v2.3 精简版 / 2026-10-17（增补 4.6 整机业务闭环）
作品名称	无人机高光谱张量农情分析系统
团队 / 赛事	光谱拾穗（Gleaners） • 2026 全国大学生嵌入式芯片与系统设计竞赛 FPGA 创新设计赛道（AMD 赛道，初级组）
适用板卡	ALINX AXU5EV-E（XCZU5EV，PS 4×A53 裸机）
工具链	Vitis / Vivado 2026.1（`D:\AMD\2026.1`） • Anaconda Python 3.14（`D:\Anaconda`）
本册范围	项目速览、上电接线、烧录下载、启动自检、SD 卡、地面站、故障速查

> **【提示】**本册只回答「怎么让系统在板上跑起来」。算法设计、性能证据与未完成项，看仓库 `README.md`、`board/*/README.md`、`report/`。

导读

板子在你手上时，按目的查表即可；每章都能单独看懂，不需要从头读。

你想做的事	翻到
先搞清楚这是个什么项目	第 1 章
第一次把板子点亮	第 2 章接线 → 第 3 章烧录 → 4.1 节 M1 自检
验证 SD / 网口 / 显示器 / 按键	4.2 节 C3 外设自检
拿加速比和性能数字	4.3 节 M2、附录 B
跑正式系统、用地面站看结果	3.3 节（GCS 下载）→ 4.5 节 → 第 7 章
让板子真的出结果（选权重、下发任务、出整幅图）	4.6 节
想用 Vitis / Vivado 打开工程、改代码	第 5 章（逐步点选 + 怎么运行）
板子不听话、报错	第 8 章故障速查

表 0-1 按目的查章

五分钟快速上手

1) **断电接线**：电源、JTAG、串口（CP2102）、网线、DP、SD 卡；SW1 拨 **0000（JTAG）**；串口开 115200 8N1 并打开日志保存（第 2 章）。
2) **先跑 M1 自检**：用第 3 章的 `dl.tcl` 下 A 路线四个文件（复位 → bit → FSBL → ELF），串口依次发 `1`、`3`、`5`，看到 `Warmup PASS`、`COMPARE PASS`、`PAIR CHECK PASS` 就说明板子通了。
3) **再验外设**：换 C3 的 bit + ELF，发 `s`（SD）、`u`（网口）、`d`（显示器）并目视确认（4.2）。
4) **看加速比**：换 M2 的 XSA/bit/ELF，发 `3` 看 `COMPARE PASS` 与 22.6× 的加速比（4.3）。
5) **跑正式系统**：按 3.3 的 E 路线下载，串口是静默的，用 DP 画面或 `ping_board.py` 判断是否 READY（4.5），PC 端再开地面站（第 6 章）。

> **【警告】三条铁律**：① 接线、插拔 SD 卡一律**断电**做；② 已经在用的 SD `journal`（`boot/jrn0.bin`、`jrn1.bin`）**不能覆盖、不能回滚**；③ 板上数字只能来自串口或地面站采集，仿真拍数、PC 推理速度都不算板上结果。

目　　录

导读 2
五分钟快速上手 2
第 1 章　项目速览 4
1.1　这是什么 4
1.2　系统组成 4
1.3　数据怎么流 5
1.4　为什么塞得进 FPGA 5
1.5　板上跑的是哪几套程序 5
1.6　目前做到哪一步 6
第 2 章　上板准备 7
2.1　清点与接线 7
2.2　接口速查 7
2.3　启动模式拨码 SW1 8
2.4　串口与网络 8
第 3 章　烧录（下载）程序 9
3.1　下载要按四步走 9
3.2　一份通用下载脚本 9
3.3　每个路线要下的四个文件 10
3.4　下载失败的常见原因 11
第 4 章　启动与自检 12
4.1　A 路线：M1 基线自检 12
4.2　B 路线：C3 外设自检 12
4.3　C 路线：M2 引擎基准 13
4.4　D 路线：M3a 整网原型（可选） 13
4.5　E 路线：GCS r1 正式系统 14
4.6　F 路线：整机业务闭环（选权重包 → 下发任务 → 出结果） 15
第 5 章　用图形界面打开工程（Vitis / Vivado） 17
5.1　先分清谁产出什么 17
5.2　Vivado：把许可证切到正确的一层 17
5.3　Vivado：打开工程 → 出比特流 → 导出 XSA 18
5.4　Vitis：打开已有的 2026.1 工作区（最快） 18
5.5　Vitis：自己从 XSA 建平台、建应用 19
5.6　在 IDE 里运行（Run / Debug） 19
5.7　打不开 / 报错时先看这里 20
第 6 章　SD 卡 21
6.1　什么时候需要卡、卡上放什么 21
6.2　journal 铁律（最容易出事的一条） 21
第 7 章　PC 地面站 22
7.1　启动 22
7.2　连接板卡的现状 22
7.3　界面分区与三个注意点 22
第 8 章　故障速查 24
8.1　先判断卡在哪一步 24
8.2　症状 → 原因 → 处理 24
8.3　复位与善后 25
附录 26
A　常量速查 26
B　已记录的板测数字 26
C　上板记录表（每轮一份） 27
D　延伸阅读与本册信息 27

第 1 章 项目速览

1.1 这是什么

一套跑在 ALINX AXU5EV-E（Zynq UltraScale+，XCZU5EV）上的无人机高光谱农情分析系统。高光谱相机拍到的每个像素不是一个颜色，而是 270 个波段的连续光谱；系统以像素为中心取 7×7×270 的邻域，用 12 层张量 Transformer 判断这个像素种的是什么作物，输出「类别 + 置信度」。PC 地面站把板卡逐行回传的结果画成分类地图，做面积统计、精度评测，并挑出需要人工核查的斑块。

边缘计算在这里很直接：原始立方体每像素 270×2 = 540 B，分类结果每像素只要 4 B，相差 135 倍。所以机上只回传结果，不回传原始光谱。

> **【提示】**当前用 WHU-Hi 公开无人机高光谱数据集的三景回放（LongKou / HongHu / HanChuan），**没有实飞，也没有接真实机载相机**；低置信度只表示「需要核查」，不是病害诊断。

1.2 系统组成

层次	承担什么	代码 / 产物
PC 地面站	显示、农情分析、精度评测、模型下发、任务控制	`gcs/`（FastAPI + 浏览器页面）、`model/`（PyTorch）
PS（A53 裸机）	读 SD、取窗与归一化量化、调度 PL、画 DP、走 UDP、计时	`src/ps/`
PL（FPGA）	张量投影、自注意力、MLP、分类头	`src/hls/`
数据	三景立方体、真值、固定划分、多版本权重	`data/scenes/<景>/`（即卡上的 `scenes/`）

表 1-1 系统组成

1.3 数据怎么流

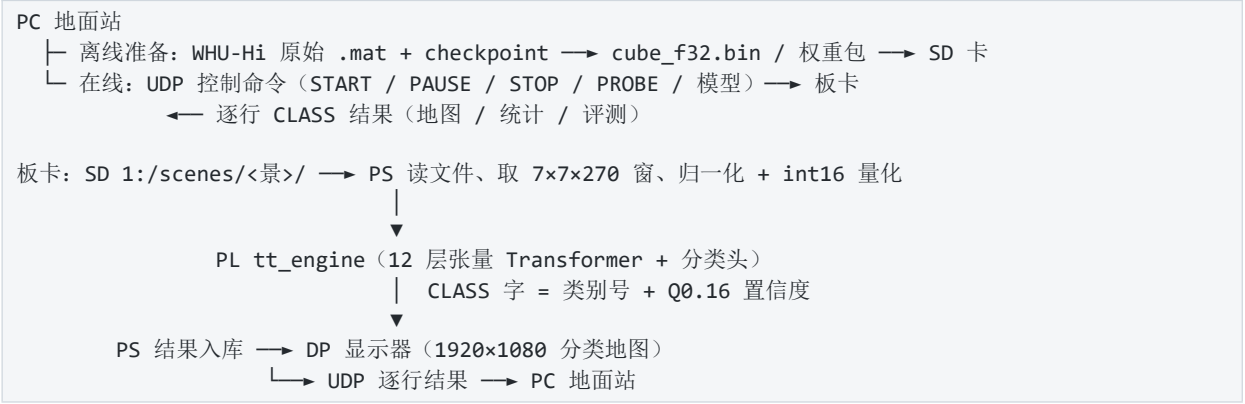


图 1-1 端到端数据流（推理全程在板上，PC 只发命令、收结果）

断网时板上照样算：任务开始、数据就绪后板卡自己按行推进，PC 断线只影响显示和控制命令。

1.4 为什么塞得进 FPGA

- 普通自注意力投影是 2646×2646 的全连接（约 700 万参数），塞不进这片 FPGA；本设计按 Tucker 形式拆成三个方向的小矩阵，一次投影只有 $7^2+7^2+54^2$ = **3014 个参数**。
- 输入 7×7×270 按波段交错切成 5 个 7×7×54 子张量，加 1 个可学习 cls token，共 **6 个 token**；两两算相似度得到 **6×6 注意力矩阵**。
- 整套结构重复 **12 层**，每层含归一化、张量投影、自注意力、残差与同样是张量投影的 MLP。
- 分类头取第 1 个光谱 token 展平成 2646 维，全连接到类别数（LongKou 9 类 / HongHu 22 类 / HanChuan 16 类）。

  置信度是 softmax 最大值，属于模型自评分，**不等于判对概率**。

1.5 板上跑的是哪几套程序

板上能跑的不止一套固件。按目的选路线，**同一路线的 XSA / bit / FSBL / ELF 必须成套**，不要跨路线拼。

路线	固件（ELF）	硬件平台（XSA）	用来做什么
A 基线自检	tt_bench_zu5ev.elf	tt_zu5ev.xsa	确认板卡、DDR、PL 通路正常，拿单 patch 基准
B 外设自检	periph_zu5ev.elf	tt_zu5ev.xsa + periph BSP	分别验证 SD / 网口 / DP / SYSMON / 按键 LED
C 引擎基准	tt_m2_zu5ev.elf（自检版 _chk.elf）	tt_m2_zu5ev.xsa	整颗引擎进 PL，与纯 PS 逐位对拍
D 加速原型	test_m3a.elf	tt_m3a_zu5ev.xsa	整网一颗 IP 的延迟与批吞吐（无菜单，研究用）
E 正式系统	gcs_r1_abi4_app.elf	tt_gcs_r1_abi4_zu5ev.xsa	SD + DP + 以太网 + ROI 任务，配地面站

表 1-2 五条运行路线

> **【提示】**第一次上板走 A；插 SD、连网线、接显示器之前，先把 A 跑通能省掉一半排查。正式系统（E）最复杂，第一次做请单独留时间。

1.6 目前做到哪一步

项目	状态	说明
M1 / M2 / M3a 基准	板测通过	2026-10-06 / 10-07，数字见附录 B
C3 外设	四项通过，一项失败	SD / UDP / DP / 按键通过；**SYSMON 仍 FAIL，未修复**
E 正式系统固件	可在板上运行	能到 READY，并经 UDP 完成过真实 ROI 任务；DP 出过画面（init_rc=0），界面内容尚未逐项目视验收
地面站连真板	被门禁挡住	本树缺 approved_elf.json，/api/board/connect 返回 409（见 7.2）
SD 独立启动（BOOT.BIN）	未做	BOOT.BIN 从未生成，当前只能 JTAG 下载（见第 6 章）

表 1-3 当前状态（2026-10-17）

第 2 章　上板准备

2.1　清点与接线

1）**断电**清点：AXU5EV-E 板卡、配套电源、JTAG USB 线、串口 USB 线（板载 CP2102）、网线、DP 线、SD 卡（≥8 GB，FAT32）、显示器。

2）**断电**接线：JTAG、串口、网线、DP 线、SD 卡全部插好；SW1 拨到 **0000（JTAG）**。

3）上电前先在 PC 上看设备管理器，确认出现 `Silicon Labs CP210x ... (COMx)`，记下 COM 号。

4）打开串口终端：**115200 / 8 / N / 1**，ASCII 文本，关闭 HEX 显示与 HEX 发送，开启原始日志保存。

5）最后接电源上电。

上电时 PL 还没加载，**风扇不转、串口没输出都是正常的**；下载完比特流风扇才转。串口长时间没字符时，先按第 8 章表 8-2 里「串口完全没输出」那一行查，不要先怀疑板子坏。

2.2　接口速查

只列本项目用到的接口；完整引脚以厂商手册 `docs/vendor/AXU5EV-E 开发板用户手册/full.md` 为准。

功能	板卡侧接口	关键参数	本项目用途
调试串口	CP2102 USB（PS_UART0）	115200 8N1、ASCII	所有程序的串口输出与菜单输入
JTAG	USB-JTAG 口	SW1 = 0000	下载 bitstream、FSBL、ELF
SD 卡	SD1 卡槽	FAT32 单分区，板内盘符 `1:/`	读立方体与权重、启动 journal
以太网	PS 网口 GEM3	KSZ9031、1000 Mbps	地面站 UDP 5010
DisplayPort	DP 输出	1920×1080@60，2 lane	板端分类地图显示
PS 按键 / LED	MIO26 / MIO40	低有效 / 高电平亮	C3 按键 LED 测试
风扇	PL 引脚 AA11	AXU5EV-E 高电平转	散热，需 PL 已加载

表 2-1　接口速查

2.3 启动模式拨码 SW1

模式	SW1	什么时候用
JTAG	0000（四位都 ON）	**本项目最常用**：日常调试与下载
QSPI	0010	从 Flash 启动（未使用）
SD	0101	SD 卡独立启动（需要 BOOT.BIN，尚未生成）
eMMC	0110	未使用

表 2-2 启动模式（上电前拨好，上电后再拨不生效）

> **【注意】**拨码 ON/OFF 与 0/1 的对应关系第一次请对着板卡丝印确认：厂商手册给的是
> JTAG=ON,ON,ON,ON；QSPI=ON,ON,OFF,ON；SD=ON,OFF,ON,OFF；eMMC=ON,OFF,OFF,ON。

2.4 串口与网络

项	值
串口	115200 / 8 / N / 1，无流控，CP2102 的 `COMx`
板卡网络	`192.168.1.10/24`，UDP `5010` 收命令
PC 网络	有线网卡静态 `192.168.1.102/24`（与板卡同网段，不要同地址）
地面站	浏览器打开 `http://127.0.0.1:8765`

表 2-3 串口与网络参数

```
# 给有线网卡配静态地址（接口名按 Get-NetAdapter 的实际名字替换）
New-NetIPAddress -InterfaceAlias "以太网" -IPAddress 192.168.1.102 -PrefixLength 24

# 验证：板卡程序跑起来且链路通时才有回包
ping 192.168.1.10
Get-NetUDPEndpoint -LocalPort 5011      # 外设 UDP 测试前确认没被占用
```

> **【注意】回包端口不固定**：板卡把回包发到「PING 包的来源端口」，PC 端的程序启动时自动绑定一个临时端口；`5011` 只是旧外设测试程序用的固定端口。所以防火墙要放行的是**你运行的那个程序**，不是某个固定端口。不要为省事关掉整个防火墙。

第 3 章 烧录（下载）程序

3.1 下载要按四步走

JTAG 下载不是「下一个 ELF 就完事」，顺序错了程序一定跑飞。

1）**系统复位**：`rst -system`，让 PS/PL 回到干净状态。

2）**烧 PL**：`fpga -file *.bit`，把 PL 配置进去（这一版的核心逻辑都在这里）。

3）**先跑 FSBL**：`fsbl.elf` 完成 PS 时钟、DDR、外设初始化，停在 `XFsbl_Exit`。

4）**再下应用 ELF** 并 `con` 运行。

> **【警告】只下 ELF 跳过 FSBL，DDR 没初始化，程序必然跑飞**——这是最常犯的一次错。另外换 ELF/XSA 前最好整机断电一次，避免沿用上一次的外设状态。

3.2 一份通用下载脚本

把下面这段存成 `D:/HSI_AgriEdge/build/host/dl.tcl`，只改「参数区」四个路径，换路线时改路径即可，其余不用动：

```
# dl.tcl —— JTAG 下载：复位 → 烧 PL → 跑 FSBL → 下应用
# 用法：  D:\AMD\2026.1\Vitis\bin\xsdb.bat D:/HSI_AgriEdge/build/host/dl.tcl

connect
source D:/AMD/2026.1/Vitis/scripts/vitis/util/zynqmp_utils.tcl

# ---------------- 参数区（只改这四行）----------------
set root D:/HSI_AgriEdge
set bit  $root/build/vivado/chk_zu5ev/chk_plat/hw/tt_zu5ev.bit
set xsa  $root/build/vivado/tt_zu5ev.xsa
set fsbl $root/build/vivado/chk_zu5ev/chk_plat/export/chk_plat/sw/chk_plat/boot/fsbl.elf
set app  $root/board/m1/tt_bench_zu5ev.elf
# ------------------------------------------------

# 1) 系统复位
targets -set -nocase -filter {name =~"APU*"}
rst -system
after 3000

# 2) 烧 PL（接了多个下载器时，先单独跑一次 targets 看 jtag_cable_name 再改过滤条件）
targets -set -filter {jtag_cable_name =~ "*"}
fpga -file $bit

# 3) 跑 FSBL 完成 PS/DDR 初始化，停在 XFsbl_Exit
targets -set -nocase -filter {name =~"APU*"}
loadhw -hw $xsa -regs -mem-ranges [list {0x80000000 0xbfffffff} \
    {0x400000000 0x5fffffff} {0x1000000000 0x7fffffffff}]
configparams force-mem-access 1
targets -set -nocase -filter {name =~ "*A53*#0"}
rst -processor
```

```
dow $fsbl
set bpx [bpadd -addr &XFsbl_Exit]
con -block -timeout 60
bpremove $bpx

# 4) 下应用并运行
targets -set -nocase -filter {name =~ "*A53*#0"}
rst -processor
dow $app
configparams force-mem-access 0
targets -set -nocase -filter {name =~ "*A53*#0"}
con
```

- 脚本来自 Vitis 生成的调试脚本，2026.1 下**没有逐条实测记录**，第一次跑请盯 xsdb 回显。
- connect 失败先查下载器与 hw_server；dow 失败先查路径是否存在（这里用正斜杠）。
- 下载完成后串口应当出现程序 banner；没有 banner 见第 8 章。想在图形界面里做同样的事，见第 5 章。

3.3 每个路线要下的四个文件

下表路径都相对工程根 D:\HSI_AgriEdge（实体在 F:\HSI_AgriEdge，用目录联接过来）。同一行的 XSA、bit、FSBL、ELF 必须来自同一套工作区。

路线	要下的四个文件（路径相对工程根 D:\HSI_AgriEdge）
A M1	XSA：build/vivado/tt_zu5ev.xsa bit：build/vivado/chk_zu5ev/chk_plat/hw/tt_zu5ev.bit FSBL：build/vivado/chk_zu5ev/chk_plat/export/chk_plat/sw/chk_plat/boot/fsbl.elf ELF：board/m1/tt_bench_zu5ev.elf
B C3	XSA：同上 tt_zu5ev.xsa bit：build/board/periph_formal_20260928/periph_plat/hw/tt_zu5ev.bit FSBL：build/board/periph_formal_20260928/periph_plat/zynqmp_fsbl/fsbl_a53.elf ELF：board/periph/periph_zu5ev.elf
C M2	XSA：build/vivado/tt_m2_zu5ev.xsa bit：build/vivado/chk_m2_zu5ev/chk_plat/hw/tt_m2_zu5ev.bit FSBL：build/vivado/chk_m2_zu5ev/chk_plat/export/chk_plat/sw/chk_plat/boot/fsbl.elf ELF：board/m2/tt_m2_zu5ev.elf（自检版 board/m2/tt_m2_zu5ev_chk.elf）
D M3a	XSA：build/vivado/tt_m3a_zu5ev.xsa bit：vitis/m3a_plat/hw/tt_m3a_zu5ev.bit FSBL：vitis/m3a_plat/zynqmp_fsbl/fsbl_a53.elf ELF：vitis/test_m3a/Debug/test_m3a.elf
E GCS	XSA：build/vivado/tt_gcs_r1_abi4_zu5ev.xsa 以下三项都在 build/host/gcs_r1_abi4_vitis/ 里： bit：gcs_r1_abi4_app/_ide/bitstream/tt_gcs_r1_abi4_zu5ev.bit FSBL：gcs_r1_abi4_plat/zynqmp_fsbl/fsbl_a53.elf PMU：gcs_r1_abi4_plat/zynqmp_pmufw/pmufw.elf ELF：gcs_r1_abi4_app/Debug/gcs_r1_abi4_app.elf

表 3-1 各路线的下载文件

> **【提示】**正式系统（E）走 Vitis 调试配置时会一并初始化 PMU；手写脚本需要自己补一段 PMU 加载，否则串口可能报 `PMU-FW is not running`。

文件	字节	SHA256 前 16 位
M1 XSA / ELF	1,892,713 / 6,197,384	`1fbcf068ca6577d2` / `edb5cf6f55b63337`
C3 ELF	1,937,960	`7b7d2c1ac47bebea`
M2 XSA / ELF / 自检 ELF	2,729,879 / 6,215,616 / 6,215,872	`81dfe6c8b153b58e` / `0e6e1b00e77d9fea` / `861a70fd94150af1`
M3a XSA / ELF	3,326,853 / 2,600,432	`9813b56322508392` / `57dfce1c6efb1ed1`
GCS XSA / ELF	3,376,874 / 3,232,488	`bc7dfb43856d2ce2` / `4cace08414f93804`

表 3-2 身份校验（下载前核一下，避免拿错版本）

3.4 下载失败的常见原因

现象	原因 / 处理
`connect` 报错或找不到设备	下载器被其它 IDE/调试会话占用；先单独起 `hw_server`，关掉旧会话
`fpga -file` 之后毫无反应	bit 与 XSA 不配套、或下载器选错；重新 `rst -system` 后再烧
下载成功但程序立刻跑飞	跳过了 FSBL，DDR/PS 没初始化；按 3.1 四步重来
`dow` 找不到文件	路径写错（本机用正斜杠）；或换了工作区没改参数区
串口一直没有 banner	COM 号/波特率/SW1；或 ELF 与本平台链接脚本不匹配

表 3-3 下载阶段排查

> **【注意】现在只能 JTAG 下载**：SD 独立启动要的 BOOT.BIN（FSBL + PMUFW + bit + 应用）从未生成，仓库里只有模板 `board/boot/gcs_round1.bif.in`，所以 SW1 保持 0000。将来要做独立启动，顺序是：确认全新介质 → 生成 journal（见第 6 章）→ `tools/gcs_boot_bundle.py` 打包 → `bootgen` 出 BOOT.BIN → 写卡 → SW1 拨 0101 → 上下电验证。

第 4 章 启动与自检

4.1 A 路线：M1 基线自检

第一次上板先跑这一条：只依赖板卡、DDR、PL 通路与权重装载，不碰 SD 卡、网口和显示器。下载完串口先打印版本横幅与 `tt_hw:` 自检行，然后等一个字符：

```
=== TT serial benchmark ===
tt_hw: ...        <- 不得出现 WARNING unaligned / ERROR PL buffer above 4 GB
(key) 1..6
```

按键	做什么	通过判据
1	单 patch（PS + PL）	`Warmup PASS`，`max|dlogit|` 在 1e-3 量级
2	单 patch，纯 PS（M0 基线）	完整 10 次全部 PASS
3	同一轮先 PS 再 PS+PL	`COMPARE PASS`，两条路径 `max|dlogit|` 差为 0
5	双 patch 流水	`PAIR CHECK PASS`，两次 patch 逐位一致

表 4-1 M1 菜单

【板测 2026-10-06】1 → 23.136 ms；2 → 356.782 ms；3 → 15.429×（PS 356.957 / PL 23.135 ms，差 0）；5 → 17.437 ms/像素。同类数字汇总见附录 B。

> **【提示】**按键 4 和 6 是 7020 时代的块推理模式，ZU5EV 上未验证，不要用。

4.2 B 路线：C3 外设自检

换成 C3 的 bit + ELF（表 3-1 的 B 行），横幅是 `=== C3 peripheral test ... ===`。**第一次务必逐项测，不要一上来按 a 全跑**——全跑分不清哪一项失败。

按键	测什么	通过判据
s	SD quick：两个文件的大小与 CRC	C3a SD PASS
S	SD 全量读测	逐文件 MB/s、耗时、CRC 全部记录
u	网口：PHY、链路、UDP 收发	C3b UDP PASS（PC 侧要同时跑 tools/periph_pc.py all）
d	DisplayPort 测试图	串口 STREAMING + 肉眼确认 1920×1080 图案
t	SYSMON 温度 / 电压	C3d SYSMON PASS
k	按键 / LED	15 s 内按 3 次都被检测到，C3e KEY/LED PASS

表 4-2 C3 菜单

【板测 2026-10-07】SD 读速 2.42 / 2.40 MB/s（全量 S 未跑）；UDP 双向 100 Mbps 零丢零错，不限速约 950～956 Mbps；DP 1920×1080@60 目视确认。

> 【警告】SYSMON 目前是 FAIL，不是「待测」：两次实测都报 PS VCC_PSINTFP 0.000 V OUT OF RANGE，PS 温度出现 229.075 / -280.231 这类值，而 PL 行正常。不要改阈值、不要跳过 PSINTFP 凑 PASS。

4.3 C 路线：M2 引擎基准

M2 把整颗 tt_engine 放进 PL，48 个投影因子一次性常驻片上；数值与 M1 逐位相同，但更快。横幅仍打印 v1.4，判别要看 tt_eng: 行。

按键	做什么	通过判据
3	同一轮先纯 PS 再 PS+PL	tt_eng: weights loaded (48 projections, ncls 9)；两组都 Warmup PASS；COMPARE PASS 且 PS-only vs PS+PL max|dlogit| 必须为 0.000e+00
5	双 patch 流水	PAIR CHECK PASS
1	可选：PS+PL 单次	多一组计时

表 4-3 M2 正式版菜单

【板测 2026-10-07】单 patch 15.776 ms（复测 15.774）、加速比 22.618×、流水 10.876 ms/像素；自检版 _chk 12 条 PASS、288 次调用 0 不一致。

> 【提示】_chk 版每次调用后用纯 PS 逐位重算，它的计时没有意义，不要当性能。

4.4 D 路线：M3a 整网原型（可选）

- 没有菜单：下载后自动跑完一发测试就停机，只看串口。

- 判据顺序：`LOAD_W PASS bytes=480576` → `M3A_CORRECTNESS PASS mismatches=0` → `M3A_BASELINE PASS` → `M3A_APP_END PASS`。
- 【板测 2026-10-07】0 mismatch；含 PS embed 3.628719 ms/patch；n=16 时 275.772 patch/s。

4.5 E 路线：GCS r1 正式系统

最终演示固件：PL 跑完整 12 层网络和分类头，PS 管 SD、DP、网络与计时，任务由 PC 下发。上板前必须先插好 SD 卡，**缺 `boot/jrn*.bin` 会直接 `S2C STOP identity` 停机**。

> **【警告】**这一版编进 ELF 时 `SERIAL_DEBUG=false`：**串口既不出字，也不收命令**。下载完串口一片安静是正常的，不要以为板子没跑起来。

串口不可用，就用下面三个手段判断它起来了，按可信度排序：

1) **DP 画面**：显示器出现 1920×1080 深色界面（标题栏、地图、任务卡、遥测卡）——最直接。
2) **UDP PING**：跑下面的小脚本，收到 PONG 说明固件在跑、网络可达。
3) **板端日志**：经 UDP QUERY 读回 `S2C BUILD / IDENTITY / READY`（地面站「刷新启动状态」做的就是这件事）。

```
板端内部打印的启动序列（写进 BoardLog，经 UDP 可读回）：
S2C BUILD app=gcs_r1_abi4 release=<64 hex> sources=<...> xsa=<...>
S2C IDENTITY source=sd-journal boot_id=<8 hex> generation=<n> abi=4
S2C MODEL_SCAN complete=<n> entries=<n> rejected=<n> rc=0
S2C DP init_rc=0 1920x1080
S2C READY boot_id=<8 hex> registry=<n> scan_ok=1 scenes=3 UDP=5010 app=gcs_r1_abi4
```

```
# ping_board.py —— 不用地面站也能确认板子活着（只用标准库）
import socket, struct, time

HDR = struct.Struct('<HBBHHI')          # magic, kind, ver, task, seq, t_us
MAGIC, PING, PONG = 0x4548, 5, 0x84
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(('', 0)); s.settimeout(0.5)
t = (time.perf_counter_ns() // 1000) & 0xffffffff
pkt = HDR.pack(MAGIC, PING, 1, 0, 0, t) + struct.pack('<I', 0x12345678)
s.sendto(pkt, ('192.168.1.10', 5010))
try:
    data, peer = s.recvfrom(2048)
except socket.timeout:
    raise SystemExit('没有回包：查网线 / IP 段 / 防火墙 / 板子是否 READY')
print('来自', peer, '类型 0x%02x' % data[2], 'PONG' if data[2] == PONG else '')
```

> **【提示】**PING 会让板卡把回包目标记成「你这个 IP:端口」，所以发 PING 的进程要占着这个 socket 别关，否则后续 STATUS / 结果包会发到没人收的端口。

4.6 F 路线：整机业务闭环（选权重包 → 下发任务 → 出结果）

E 路线下载完，DP 亮起、板卡也 READY，但画面上一直是「等待任务 / 尚未选择模型」，任务卡写着「等待地面任务」。**这不是故障，也不是没跑起来**：这一版固件自己不选权重包、也不产生任务，`WACT`（选版本）和 `START`（下发任务）都要由 PC 端发出，板卡只是等着收。

地面站起不来的情况（原因见 7.2），用仓库里这四个**只用标准库**的脚本，效果等价：

脚本	做什么	什么时候用
`tools/board_model.py`	只选权重包（等价在地面站点一下场景/版本）	先让 DP 上出现模型名与 CRC
`tools/demo_roi.py`	一键：选权重 → 下发一块 ROI → 收结果、逐像素打印	最快的「板子真的在算」证明
`tools/gcs_mini.py`	窗口版小地面站：整幅或 ROI，实时分类图 + 类别统计，可存 PNG / CSV	要出整幅图与统计
`tools/s2c_roi_client.py`	严格验收：5×3 结果与 2026-10-09 板测参考逐位比对	回归、交付证据

表 4-4 业务闭环工具

```
# 1) 只选权重包（LongKou v2，wcrc = 27231983）
D:\Anaconda\python.exe D:\HSI_AgriEdge\tools\board_model.py

# 2) 一键 5×3，打印 15 个像素
D:\Anaconda\python.exe D:\HSI_AgriEdge\tools\demo_roi.py

# 3) 窗口版小地面站：选场景 → 选区域（整幅 / 64x64 / ...）→ 点「开始」
D:\Anaconda\python.exe D:\HSI_AgriEdge\tools\gcs_mini.py
```

`demo_roi.py` 的实测输出（节选）：

```
[1/4] 连板卡 192.168.1.10:5010（PING …）
      boot_id=228ce0f4  当前模型 wcrc=27231983  状态 state=3  任务=65265
[3/4] 下发 ROI 任务 3 行 × 5 列（(0,0) 起点，mode=1）
      ACK type=0x01 code=0 seq=2 value=0x00000000
      STATUS state=1 done=0/15 front=0
      ROW y=0: A58F0000 92170005 AED80000 F5ED0000 F8760000
      ...
      DONE  done=15/15  tx_fail=0 tx_retx=0 rx_bad=0 rx_gap=0 rx_dup=0
```

这 15 个字与 2026-10-09 板测记录（`report/system_s1b/20261009_app_board_r1/uart_attempt_03_user.txt`）**逐位一致**，`s2c_roi_client.py` 判 `PASS`，证据目录 `build/host/board_demo_<时间戳>/`（`summary.json` + `packets.jsonl`）。

y\x	x=0	x=1	x=2	x=3	x=4
0	A58F0000	92170005	AED80000	F5ED0000	F8760000
1	F6110000	E48B0000	FA0C0000	FF540000	FF700000
2	F5880000	F08F0000	FBF60000	FF610000	FED10000

表 4-5 5×3 ROI 的 15 个字（低字节类别，高 16 位 Q0.16 置信度）

> **【说明】速度**：板上实测 4.42 ms/像素（int16 引擎）。换算：64×64 ≈ 18 s；整幅 LongKou 400×550 ≈ 16 min；HongHu 475×940 ≈ 35 min、HanChuan 303×1217 ≈ 27 min。整幅跑起来后 DP 按行推进刷新，不用等跑完才看得到东西。

【板测 2026-10-17】整幅 LongKou 一次跑完：**220,000 像素 / 550 行 / 9 类全部出现**，用时约 15.5 分钟；分类图与统计存在 `build/host/gcs_mini_<时间戳>/`（`class_map.png`、`class_stats.csv`、`summary.json`）。完整记录见 `docs/board_demo_20261017_migration_check.md`。

> **【警告】会话号**：板卡上电后绑定第一个 PING 的会话号（脚本默认 `0x5a5a0001`），此后别的会话号一律不回包，所以别让两个工具用不同会话号抢同一块板卡；工具报「没有回包」时先想到这一条，或换 `--session`。

任务号在同一上电周期内不能重复（板卡记着用过的任务号，重复下发的回 `ACK code 9`）。上面几个脚本都用时间戳生成任务号，不用管；自己写客户端时要注意。

第 5 章 用图形界面打开工程（Vitis / Vivado）

只想让板子跑起来，不用打开 IDE：第 3 章的脚本一条命令就做完「复位 → 比特流 → FSBL → 应用」。这一章是给「想看看工程长什么样、想改代码、想自己重建」的时候用的，按顺序照着点即可。

5.1 先分清谁产出什么

工具	产出	用在哪
Vivado	比特流 .bit、硬件描述 .xsa	PL 的配置；.xsa 给 Vitis 建平台
Vitis	FSBL、PMU 固件、BSP、应用 .elf	PS 侧启动与运行
xsdb（命令行）	把上面几样按顺序写进板子	第 3 章的脚本，最快

表 5-1 三个工具的分工

【警告】本机只装了 2026.1。仓库里 vitis\、build/host/gcs_r1_abi4_vitis\ 是 2020.2 建的工程，2026.1 打不开（只能看源码）；能直接打开的是 build/board/*_20261016* 这几个 2026.1 工作区。

5.2 Vivado：把许可证切到正确的一层

本机的 Xilinx 许可证文件里同时有两层：ALVEO（只覆盖 Alveo 加速卡）和 BASIC（覆盖 Zynq UltraScale+ 等普通器件）。Vivado 默认挑 ALVEO 那一层，于是打开本工程会报 Specified part could not be found——器件文件其实装了，是选错了许可证层。

1) 用仓库脚本启动 Vivado：tools\vivado\launch_vivado.bat。它会把 BASIC 层单独写一份副本到 build/host/lic/vivado_basic.lic，再用 XILINXD_LICENSE_FILE 指过去（原许可证文件不动）。
2) 想手工来：先 set XILINXD_LICENSE_FILE=D:\HSI_AgriEdge\build\host\lic\vivado_basic.lic（副本由 tools\vivado\make_basic_license.py 生成），再开 Vivado。
3) 验证：Vivado 的 Tcl Console 里敲 get_parts -quiet *zu5ev*，能列出 26 个 ZU5EV 器件就对了。

【提示】副本只保留 BASIC 层；要看 Alveo 器件时不设这个环境变量、直接开 Vivado 即可。给别的机器用同一条命令也适用（脚本会自动找许可证文件）。

5.3 Vivado：打开工程 → 出比特流 → 导出 XSA

仓库里四个 Vivado 工程都是 2020.2 建的，而且记录的是车规器件 xazu5ev-sfvc784-1-i（本机器件库里没有）；实物板子是 XCZU5EV-2SFVC784I，对应 xczu5ev-sfvc784-2-i。所以已经准备好四份 2026.1 副本（复制 + 升级 + 改好器件，原件没动）：

路线	打开这份 (build/host/vivado2026/ 下)	对应原件（build/vivado/ 下）
A M1 / B C3	tt_zu5ev/tt_zu5ev.xpr	tt_zu5ev/
C M2	tt_m2_zu5ev/tt_m2_zu5ev.xpr	tt_m2_zu5ev/
D M3a	tt_m3a_zu5ev/tt_m3a_zu5ev.xpr	tt_m3a_zu5ev/
E GCS	tt_gcs_r1_abi4_zu5ev/tt_gcs_r1_abi4_zu5ev.xpr	tt_gcs_r1_abi4_zu5ev/

表 5-3 可直接在 2026.1 打开的工程副本

1) 启动：tools\vivado\launch_vivado.bat "build\host\vivado2026\tt_zu5ev\tt_zu5ev.xpr"（不带参数就是只开 Vivado）。
2) 左侧 Flow Navigator 最下面点 Generate Bitstream（综合 + 实现 + 出比特流，十几分钟起）。
3) 完成后比特流在 tt_zu5ev.runs\impl_1\ 下；再点 File → Export → Export Hardware，勾 Include bitstream，输出到 D:\HSI_AgriEdge\build\vivado\，得到新的 .xsa。
4) 要按原件重新做一份副本：pwsh -File tools\vivado\prepare_project_copy.ps1 -Name tt_zu5ev（复制 → 升级 → 改器件，原件不动）。
5) 直接打开原件也行：2026.1 会提示升级，并在「器件不可用」的弹窗里让你选器件——选 xczu5ev-sfvc784-2-i（与板卡一致）继续即可。

> **【警告】别覆盖已经在用的产物**：build/vivado/tt_zu5ev.xsa 这几个是上板验证过的，哈希写在表 3-2 里，覆盖就对不上号了。要重建就导出成新文件名，或另建目录。

5.4 Vitis：打开已有的 2026.1 工作区（最快）

1) 最省事：tools\vitis\open_workspace.bat M1（参数可写 M1 / C3 / M2 / M3A，或直接写工作区目录名）。等价的命令行是 D:\AMD\2026.1\Vitis\bin\vitis.bat -w D:\HSI_AgriEdge\build\board\chk_m1_20261016b；也可以从开始菜单 Vitis 2026.1 启动，首次弹出的工作区选择框里选这个目录。
2) 左侧 Vitis Components 面板会出现两个组件：chk_plat（平台，含 FSBL / PMUFW / BSP）和 tt_bench_zu5ev（应用）。
3) 点应用组件 → 右侧 Flow Navigator 里点 Build 编译；日志在下方 Terminal。
4) 产物就是第 3 章要下的 ELF：...\tt_bench_zu5ev\build\tt_bench_zu5ev.elf。

路线	工作区（都在 build/board/ 下）	平台组件	应用组件
A M1	chk_m1_20261016b	chk_plat	tt_bench_zu5ev
B C3	periph_ws_20261016	periph_plat	periph_zu5ev
C M2	chk_m2_20261016b	chk_plat	tt_m2_zu5ev、tt_m2_zu5ev_chk
D M3a	chk_m3a_20261016	chk_plat	test_m3a

表 5-2 四个能直接打开的 2026.1 工作区

编译产物统一放在「应用组件目录 / build / 应用名.elf」：上面 A 路线就是 tt_bench_zu5ev 组件下的 build/tt_bench_zu5ev.elf。

> 【提示】平台名都叫 chk_plat，但它们属于不同工作区、不同 XSA——认目录，不认名字。

5.5 Vitis：自己从 XSA 建平台、建应用

要改硬件配置、换 BSP 库时才需要。点法和建普通工程一样：

1) 左侧 Vitis Components 面板空白处右键 → New Component（或菜单 File → New Component）。
2) 类型选 Platform，名字随意（例如 m1_plat）。
3) 配置页里：XSA 选 D:\HSI_AgriEdge\build\vivado\tt_zu5ev.xsa；处理器 psu_cortexa53_0；OS 选 standalone；Flow 选 Embedded。
4) BSP 库：要 SD 勾 xilffs，要网口勾 lwip220，DP 用 dppsu（2026.1 默认）。
5) Finish 后点 Build，看到 FSBL / PMUFW / BSP 都生成即成功。
6) 再建应用：New Component → Application → 选刚建的平台、domain standalone_domain、模板 Empty Application → 把 src/ps/ 的源码拷进组件 src/ → Build。

> 【警告】手工建会遇到三个 2026.1 的坑，仓库脚本已经自动处理了：① 安装里的 lwip220.yaml 带 BOM，lwIP 库直接不可用；② 2026.1 不再把 tt_engine / tucker_proj 的 HLS 驱动编进 BSP；③ 2026.1 不再导出 XPAR_* 旧宏。所以「一次编过」最省事的做法是用 tools/vitis_py\build_*.py（命令见附录 A），图形界面更适合看和改配置。

5.6 在 IDE 里运行（Run / Debug）

跑一次只要说清四件事：下哪个比特流、用哪个 FSBL 初始化、下哪个 ELF、要不要先复位。

1) 在 Vitis Components 里选中应用组件 → 上方 Flow Navigator 点 Debug（只想跑不调试就点 Run），第一次会打开启动配置。
2) Target Connection 选你的下载器（只有一台时选 Local 即可）。

3) Bitstream File 选对应平台的比特流：2026.1 工作区在 <平台>\hw\sdt\tt_zu5ev.bit，2020.2 工作区在 <平台>\hw\tt_zu5ev.bit。

4) ZU Initialization 选 Using FSBL，FSBL File 选 <平台>\zynqmp_fsbl\build\fsbl.elf（2020.2 工作区是 ...\export\...\boot\fsbl.elf）。

5) 勾上 Reset Entire System 与 Program Device；Download Elf 里确认是本应用的 ELF。

6) 点 Debug 启动；要让程序继续跑，点工具栏的 ▶（Resume）。串口表现见第 4 章对应路线。

要做的事	界面位置（Vitis 2026.1）	对应脚本（第 3 章）
系统复位	Target Setup → Reset Entire System	rst -system
烧 PL	Target Setup → Program Device + Bitstream File	fpga -file $bit
PS / DDR 初始化	ZU Initialization → Using FSBL + FSBL File	dow $fsbl，跑到 XFsbl_Exit
下应用	Target Setup → Download Elf	dow $app

表 5-4 界面操作和脚本的对应关系

【提示】界面文字在不同小版本里可能略有差异（例如 Run / Debug、ZU Initialization 的写法）；**只想跑一次、不想配界面，就直接用第 3.2 节的脚本**。

5.7 打不开 / 报错时先看这里

现象	原因	处理
打开 .xpr 报 part not found 或 license 不覆盖器件	许可证默认选中了 ALVEO 层	用 tools\vivado\launch_vivado.bat 启动（见 5.2）
打开副本后仍提示器件不对	选到了车规器件或没改器件	器件选 xczu5ev-sfvc784-2-i；或重跑 prepare_project_copy.ps1
2020.2 工程升级后综合 / 实现报错	跨版本差异	先备份；改用 build/vivado 里的 tcl 重建工程
Vitis 里打开 vitis\ 说无法识别	那是 2020.2 的经典工作区	2026.1 只能打开 build/board/*_20261016*
Build 报找不到 lwip220 / xtt_engine.h / XPAR_*	三个已知 2026.1 差异（见 5.5）	用 tools/vitis_py\build_*.py 重建
Build 报编码错误或找不到文件	路径含中文	一律从 D:\HSI_AgriEdge 操作
Debug 卡在 connect	下载器被占用，或 hw_server 没起	关掉其它 IDE / 调试会话，先起 hw_server

表 5-5 图形界面常见问题

【提示】图形界面这条路能走通，但本仓库唯一被完整跑通过的下板方式还是**第 3 章的 xsdb 脚本**；界面这边出了问题，先回到脚本把板子跑起来，再回来查 IDE。

第 6 章 SD 卡

6.1 什么时候需要卡、卡上放什么

用途	卡上必须有	说明
C3 外设自检	`scenes/longkou/cube_i16.bin`（122,800,384 B） `scenes/longkou/weights/v2/weights_engine.bin`（480,576 B）	第二个文件用来验证长文件名；拷完用 `tools/periph_pc.py crc --root E:/scenes` 核对
正式系统（GCS）	`scenes/` 三景 45 个文件，共 1,150,149,380 B `boot/jrn0.bin`、`boot/jrn1.bin` 各 64 B	清单见 `build/host/gcs_r1_abi4_user/sd_required_layout_v3.txt`，每个文件带 SHA256
卡格式	单分区 FAT32（MBR 类型 0x0C）	**exFAT 不行**，驱动会报 `FR_NO_FILESYSTEM`

表 6-1 两种卡

SD 卡本身不是必需项：M1 / M2 / M3a 基准完全不读卡。只有 C3 的 `s/S` 和正式系统才要。

6.2 journal 铁律（最容易出事的一条）

> **【警告】**journal 是板端用来证明「这是一次真实启动、不是回滚或克隆」的双槽记录。**已经在用的卡不能覆盖、不能回滚、不能用新生成的替换**，动了它板子会停机或身份错乱。

- 判断手里这张卡是不是「已安装」：卡上有 `boot/` 目录就是，按已安装处理，别再碰。
- 只有确认是全新安装介质时才生成新 journal：`tools/gcs_boot_provision.py --new-installation --out <新目录>`。它**只生成本地文件、不写卡**，你再把 `jrn0.bin`、`jrn1.bin` 复制到卡上。
- 旧卡的 journal 与启动日志属于原始证据，保留不删。

第 7 章 PC 地面站

浏览器里的操作台：连板卡、下发任务、看分类地图、做农情分析与评测、导出证据。它本身不参与推理，也不读真值去补结果。

7.1 启动

```
# 首次安装依赖（本机目前缺 fastapi / torch / pyserial）
D:\Anaconda\python.exe -m pip install fastapi "uvicorn[standard]" pyserial
D:\Anaconda\python.exe -m pip install torch --index-url
https://download.pytorch.org/whl/cpu

# 启动（默认 http://127.0.0.1:8765，1.5 s 后自动开浏览器）
D:\Anaconda\python.exe tools\gcs_roi.py serve --port 8765
```

> **【提示】**`gcs_roi.py serve` 会先探测端口：8765 上已有 `service=gcs-s2c-roi` 时**直接复用**，所以「启动」秒返回不代表起了新进程。

7.2 连接板卡的现状

> **【警告】本树连不上真板，是发布门禁拦的，不是 bug。** `/api/board/connect` 先做发布审查，要求默认发布包里存在 `approved_elf.json`；本树没有这个文件，于是返回 HTTP 409。被部署成 v8 的运行树（含审查过的 ELF 与冻结包）不在本机。

三条现实选择：① 先不连接，用 4.5 的 PING 脚本和 `tools/s2c_roi_client.py` 验证板子，地面站只当界面预览；② 用 `tools/gcs_release_elf.py` 对这份 ELF 重新做发布审查并注册；③ 把 v8 冻结包放回 `build/host/` 并把服务默认包指过去。**不要改门禁代码或伪造 `approved_elf.json` 来「连上」。**

要「在 PC 上看结果、做操作」而不修发布门禁，用 4.6 的 `tools/gcs_mini.py`：它只走 UDP 协议，不经过地面站的发布审查，能直接下发任务、实时画分类图、统计各类占比、导出 PNG / CSV。

7.3 界面分区与三个注意点

- 顶栏选景与权重版本、切二维 / 三维；中间是分类地图（左侧工具条做平移探查、框选区域、绘制地块）；右栏是任务卡与遥测；底部页签是单点探查、农情分析、地块台账、精度指标、链路与下发、历史对比、日志。
- 暂停与停止**只在完整行边界生效**，ACK 只代表命令收到，以板上 STATUS 为准。

- 「评测」在板卡数据源下被禁用——板上没有真值输入。

第 8 章 故障速查

8.1 先判断卡在哪一步

阶段	看到什么
① 上电	电源灯、串口设备是否出现
② 下载	xsdb 是否报连接或下载错误
③ 启动	串口有没有 banner、`tt_hw:` / `tt_eng:` / `S2C` 行
④ 外设	SD / 网口 / DP / SYSMON / 按键
⑤ 任务	地面站能否连接、任务能否开始、结果是否连续

表 8-1 阶段定位

8.2 症状 → 原因 → 处理

现象	可能原因	处理
上电没反应	电源未接好、适配器不匹配	断电重接，核对规格，换电源试
风扇不转	PL 未加载（风扇由 PL AA11 驱动）；或实物是 B 型（极性相反）	先确认 bit 真的下进去了；再按丝印核实板型
串口完全没输出	设备管理器没有 COM 口（CP210x 驱动未装、线只供电）、SW1 不是 0000、COM 号错、没跑 FSBL	装厂商驱动换数据线；确认 SW1、COM 号、115200 8N1，再确认 FSBL 跑过
串口乱码	波特率不一致、开了 HEX 显示	统一 115200 8N1，关闭 HEX
下载成功但程序跑飞	跳过了 FSBL，DDR/PS 未初始化	按 3.1 四步重来
`ERROR PL buffer above 4 GB` `WARNING unaligned`	链接脚本把 DMA 缓冲放到高地址，或缓冲未 64 B 对齐	把段放回 `psu_ddr_0_MEM_0`（<4 GiB），缓冲 64 B 对齐
`tt_eng: ERROR LOAD_W`	权重包头不对（err=8）或 IP/寄存器头不配套（version≠3）	用对应 XSA 新建平台；换 IP 版本后重建平台，不要只 Update Hardware Specification
`tt_eng` 卡住不返回	第一次调用就卡：PL 未正确初始化	用 `_chk` 自检版定位；`mrd` 读 CTRL 寄存器；必要时外部复位 PL
`PMU-FW is not running`	只下了 bit / FSBL / ELF，没载 PMUFW	在 Vitis 调试配置里初始化 PMU 后重下
SD mount 失败	卡不是 FAT32；或应用域 `use_lfn=0`	格式化为 FAT32；应用域参数 `use_lfn=1`
SD 能列出文件但读失败	介质或文件系统损坏；PC 只核对了元数据	`periph_pc.py crc --root` 实读核对，用已核原件重拷，不要自动格式化
UDP 无回包	IP 不同段、防火墙、端口占用、PHY 链路未起	PC 设 `192.168.1.102/24`，放行测试程序，查 `Get-NetUDPEndpoint`
DP 无画面	显示器没切到 DP、链路训练失败、热下载后的初始化时序	显示器先开机切 DP，直连 DP 线；整机断电重启后重下同一 ELF
有 `STREAMING` 但画面异常	像素格式 / 字节序，或驱动不是 dppsu 1.3	核对 rate code 0x14、2 lane、148.5 MHz
SYSMON FAIL	PS 通道采样异常（`VCC_PSINTFP` 读 0 V、温度读数不合理）	**已知未修复项**：不改阈值、不跳过 PSINTFP

现象	可能原因	处理
地面站连不上板卡	发布门禁缺 `approved_elf.json`（本树现状），或板端未 READY	见 7.2；先确认板端已 READY
板端不是 READY	`S2C STOP …` 停机、SD 缺文件、BUILD 行被截断	读原字与 rc，补齐 SD 文件；不要放宽门禁
任务开始后中途停	启动握手超时（5 s）、30 s 无包、非空闲状态	重连、确认没有旧任务、查网线；保留首个失败
Vivado 报 part not found / license 不覆盖器件	许可证默认选中 ALVEO 层（只覆盖 Alveo 卡）	用 `tools\vivado\launch_vivado.bat` 启动；只做下载不需要 Vivado
HLS `set_msg_logfile` 报错 Vitis `codec can't decode 0xd5`	路径或用户名含中文	一律从 `D:\HSI_AgriEdge` 构建（ASCII 路径）
DP 一直「等待任务 / 尚未选择模型」	这一版固件不自带任务，选权重与下发任务由 PC 端做；不是死机	跑 `tools/demo_roi.py` 或 `tools/gcs_mini.py`（4.6）
工具 PING 没回包，但板子明明是好的	会话号不是板卡绑定的那个（板卡只认上电后第一个 PING 的会话号）	用 `--session` 换成当初绑定的会话号；别让两个工具用不同会话号抢同一块板卡
START 被回 `ACK code 9`	任务号在同一上电周期内重复，或板卡还有任务在跑（`code 8` = 参数越界）	换一个任务号；先发 STOP 等 `state` 变 0 再下发

表 8-2 症状 / 原因 / 处理

8.3 复位与善后

1) **先看串口**：程序自己停了会打印 `S2C STOP …` / `S2C HALTED …`，那行就是根因。
2) DMA / LOAD_W 类故障会**永久停车**，只能外部复位 PL 并重启应用，软件 init 清不掉。
3) JTAG 复位：xsdb 里 `rst -system`，再按 3.1 四步重下。
4) 失败原样保留：不清日志、不靠重启掩盖、不把「重试一次好了」写成 PASS。

附录

A 常量速查

项	值
工程根	D:\HSI_AgriEdge（目录联接 → F:\HSI_AgriEdge，必须用 ASCII 路径构建）
工具链 / Python	D:\AMD\2026.1 / D:\Anaconda\python.exe（3.14）
串口	115200 8N1，CP2102 的 COMx
板卡网络	192.168.1.10/24，UDP 5010 收命令，MAC 00:0a:35:00:01:02
PC 网络 / 回包端口	192.168.1.102/24 / 由程序自动绑定（回包发到 PING 源端口）
地面站 / hw_server	http://127.0.0.1:8765 / tcp:127.0.0.1:3121
SD	板内盘符 1:/，数据在 scenes/<景>/，journal 在 boot/
LongKou v2 引擎权重	480,576 B，WCRC 常量 27231983
PL 时钟 / PS 计时基准	200 MHz / 33.333 MHz 计数器（COUNTS_PER_SECOND）

表 A-1 常量与路径

B 已记录的板测数字

平台 / 项目	结果	日期
M1 单 patch（PS+PL）	23.136 ms，误差 1.149e-03，10/10 PASS	10-06
M1 纯 PS（M0 基线）	356.782 ms	10-06
M1 同轮加速比	15.429×（PS 356.957 / PL 23.135 ms，两条路径差 0）	10-06
M1 双 patch 流水	17.437 ms/像素（顺序 23.130 ms，1.326×）	10-06
M2 单 patch / 加速比	15.776 ms（复测 15.774）/ 22.618×（复测 22.640×）	10-07
M2 双 patch 流水	10.876 ms/像素（顺序 15.760 ms，1.449×）	10-07
M3a 单 patch / 批吞吐	3.628719 ms（含 PS embed）/ 275.772 patch/s（n=16）	10-07
C3 SD / UDP / DP	2.42、2.40 MB/s / 100 Mbps 双向零丢（不限速 950～956 Mbps）/ 画帧 24.8 ms	10-07

平台 / 项目	结果	日期
GCS 整景（LongKou 22 万像素）	「900 多秒」量级（仅定性记录，精确值未留存）	10-10
GCS 板上精度（LongKou Test100）	203,642 像素：OA 93.8362% / AA 92.7284% / Kappa 0.920108	10-10

表 B-1 可对外引用的板上数字

- 板上数字**只能来自串口或地面站采集**；HLS cosim 的 1,083,888 拍 ≈ 5.42 ms @200 MHz 是仿真值，不能当板上实测。
- 「流水 ms/像素」比「单次 ms」小，两者不能互算加速比；LOAD_W 是装载权重的一次性开销，不能摊到每像素。
- 没有记录的数字（功耗、整板能耗、连续长时间稳定性）不要补写。

C 上板记录表（每轮一份）

项目	记录内容
日期 / 板卡 / 拨码	AXU5EV-E，SW1 = _____
本轮身份	路线（A～E）、XSA 与 ELF 的 SHA256 前 16 位
原始证据	串口原文日志路径、PC 采集 JSON、DP/LED 照片
结论	PASS / FAIL / 部分（写明哪一项没测）

表 C-1 上板记录（没测的写「未测」，失败保留原字，不要重跑一次通过就抹掉第一次的失败）

D 延伸阅读与本册信息

想知道什么	看哪份文件
项目总览、目录结构	README.md、CLAUDE.md
方案与执行计划	docs/PLAN.md
板卡硬件细节（MIO / 引脚 / 拨码）	docs/BOARD_AXU5EV_E.md、厂商手册 docs/vendor/…/full.md
上板检查清单	board/CHECKLIST.md、board/m1/README.md、board/m2/README.md、board/periph/README.md
工具链迁移与许可证	docs/vitis2026_migration.md
性能与证据	report/perf_m1.md、perf_m2.md、perf_m3.md、perf_c3.md、report/evidence.md
地面站功能说明	docs/gcs_guide.md（界面细节以当前页面为准）

表 D-1 文档索引

本册 v2.3 精简版（2026-10-17）取代 v2.2 / v2.1 / v2.0 / v1.0，早期版本保留作存档；对应代码基线 git e743ce8。新增 4.6「整机业务闭环」与故障速查三行。代码或产物变化后，先改对应章节，再核附录 B 的数字。